

I have an application that has a single mod bus server mod bus TCP server in that single mod bus TCP server is on a single industrial PC and that single mod buzz TCP server has around 10,000 or more data points based on hundreds of different devices that have that data and it's a simulator based on natural gas reciprocators that are in an island in micro grid so there are 40 GEN sets generators Jen sets each GEN set is the same and Jensen has the same mod bus TCP data and what I need to do is build a an additional simulator that sits on a layer above the industrial PC that actually breaks out the MOD buzz TCP data from the industrial PC into each individual unit and stimulates it so for example, if there are only 15 of the 40 GEN sets online and operating then only 15 mod bus TCP servers are deployed and it's gonna be running on a Kuper Brunetti's cluster so it's gonna be a single IP address the same IP address for each simulated device but we can use a different mod bus TCPIP port so for example Jen sent one is on the dedicated IP address at Port 505 then Jen said two is at Port 506 with the same IP address, etc. give me some options of how this could be built and deployed either with open source predefined software or customized for example I had a python script that simulated modbus TCP data before I also know there's mod SIM a software mod SIM and how could this be built and deployed?

I'll lay out viable architectures first, then concrete deployment options for Kubernetes, including where Python/custom code makes sense versus off-the-shelf Modbus simulators.

Thought for 7s >

Best practical options:

Option 1 — Custom Python simulator per genset

Use **PyModbus** or **pyModbusTCP**. Best fit for your case.

Architecture:

```
Existing industrial PC simulator
Single Modbus TCP server
~10,000 points
    |
    | polling / mapping layer
    v
Kubernetes simulator layer
    |
    |-- Genset 01 Modbus TCP server :505
    |-- Genset 02 Modbus TCP server :506
    |-- Genset 03 Modbus TCP server :507
    ...
    |-- Genset 40 Modbus TCP server :544
```

Each pod/server exposes only that genset's register map. If only 15 gensets are online, only 15 servers are started or marked available.

Use this when you need:

- Dynamic online/offline behavior
- Register values calculated from the upstream 10,000-point simulator
- Same register map repeated 40 times
- Realistic state behavior: start, sync, load, trip, cooldown, alarm, etc.
- Kubernetes-native deployment

PyModbus supports TCP servers and asyncio-based servers; its simulator is configurable, but the v4 simulator docs still warn that the new simulator is work-in-progress, so I would use the normal PyModbus server API for production-style testing, not rely on the packaged simulator UI. [PyModbus +1](#)

Option 2 — One Python process hosting 40 ports

Instead of 40 pods, run one container with 40 Modbus TCP listeners:

```
simulator-service IP
:505  genset_01
:506  genset_02
:507  genset_03
...
:544  genset_40
```

This is probably your best rev0.

Pros:

- Simple deployment
- One container
- One config file
- Easy to map upstream registers to virtual gensets
- Easy to enable/disable individual gensets
- Lower Kubernetes overhead

Cons:

- One process failure affects all simulated units
- Less realistic than one pod per device
- Scaling is less clean

For rev0, I would do this first.

Option 3 — One pod per genset

Deploy 40 identical containers using a Helm chart or StatefulSet.

Each pod gets environment variables:

```
GENSET_ID=01
LISTEN_PORT=505
UPSTREAM_HOST=industrial-pc-sim
UPSTREAM_PORT=502
REGISTER_OFFSET=0
```

Genset 2:

```
GENSET_ID=02
LISTEN_PORT=506
REGISTER_OFFSET=250
```

Pros:

- Clean device isolation
- Easy to kill/start individual simulated gensets
- More realistic failure testing
- Better for future non-genset devices: BESS, sync condenser, relay, breaker, etc.

Cons:

- More Kubernetes objects
- More config management
- More networking setup

Use this once rev0 works.

Option 4 — ModbusPal

ModbusPal is a Java Modbus slave simulator with TCP/IP support and scripting for animated register values. [ModbusPal](#) +1

Good for:

- Manual testing
- Lab demos
- Basic simulated slaves
- Scripted register animation

Bad fit for your final use case:

- Not Kubernetes-first
- Java GUI/tooling baggage
- Harder to dynamically mirror and split an upstream 10,000-point simulator
- Less clean for 40 dynamic device instances

I would not choose this as the main architecture.

Option 5 — diagslave

diagslave is a free command-line Modbus slave simulator available for Linux, Windows, QNX, Raspberry Pi, and ARM Linux. [Modbus Driver](#)

Good for:

Bash

```
diagslave -m tcp -a 1 -p 505
```

Useful for:

- Smoke testing ports
- Proving Kubernetes service exposure
- Basic Modbus client validation

Bad fit for:

- 10,000 mapped points

- Dynamic genset state models
- Upstream data transformation
- Realistic operational behavior

Use it as a test utility, not the simulator core.

Recommended build

Rev0

Build a **single Python container** using PyModbus.

It should:

1. Poll the existing industrial PC Modbus server.
2. Maintain an internal object model:

```
Plant
  Genset[1..40]
    status
    breaker
    voltage
    frequency
    kW
    kvar
    alarms
    permissives
    Modbus register map
```

3. Start TCP listeners:

```
505 = Genset 01
506 = Genset 02
...
544 = Genset 40
```

4. Only expose active gensets, or expose all and return offline/status-failed values for inactive ones.

Rev1

Move to **one pod per simulated device** if you need better isolation.

Rev2

Add non-genset simulated devices:

```
Sync condenser
BESS
MV breaker
LV black start genset
SEL relay
Load bank
FEP
Master controller
```

Kubernetes deployment shape

Use one service with multiple named ports:

YAML

```
ports:
- name: gen01
  port: 505
  targetPort: 505
- name: gen02
  port: 506
  targetPort: 506
- name: gen03
  port: 507
  targetPort: 507
```

Or use NodePort/LoadBalancer if external SCADA clients need to reach it.

My recommendation

Do **custom Python + PyModbus**, not ModSim/ModbusPal, for the main simulator.

Reason: your requirement is not just "simulate Modbus." It is:

```
Split one large plant-level simulator into 40 device-level Modbus TCP endpoints,
map repeated equipment models,
control online/offline behavior,
and support realistic microgrid state behavior.
```

That is a custom data-modeling problem. Off-the-shelf tools will slow you down once you need realism.



So an additional caveat I need to build this in my environment, which is Windows 11 with one of the various IDE that I work with cursor AIVS code, etc. and I need to get the simulators working and then there is my colleague who is on this server side that configures the Cabernets cluster deploys everything etc. so it needs to be ready for him and I'm not gonna be able to work on the Kuper's cluster I don't know how to work with all of that so with that let me know if there's any revisions to your previous message so I need to basically build something that I can then turn over to him and it's gonna be as seamless as possible and straightforward for him to deploy them. I know they're gonna be some configuring he has to do but like I don't know if doctor or I don't know some however you can like containerize from Windows 11 think can be given to Kuper Burnett's or maybe there's a little bit of reconfiguration is fine.

Thought for a couple of seconds >

Yes — revise the recommendation:

Best path for your situation

Build it locally on **Windows 11** as a **Dockerized Python Modbus simulator**, then hand your colleague:

```
/project-folder
  app/
  config/
  Dockerfile
  docker-compose.yml
  requirements.txt
  README.md
  k8s-example.yaml
```

Your job: make it work in Windows.

His job: deploy the same container into Kubernetes.

Recommended architecture

Local Windows development

Use:

```
Windows 11  
Cursor / VS Code  
Python  
PyModbus  
Docker Desktop  
docker-compose
```

You run and test it locally like this:

PowerShell

```
docker compose up --build
```

Then test ports:

PowerShell

```
localhost:505 = Genset 01  
localhost:506 = Genset 02  
localhost:507 = Genset 03  
...
```

Once it works, your colleague can take the same Docker image and deploy it to Kubernetes.

Revised deployment model

For your case, I would not start with 40 separate containers.

Start with:

```
One Docker container  
One Python app  
40 Modbus TCP servers inside it  
Each server uses a different TCP port
```

Example:


```
Genset 01 -> port 505
Genset 02 -> port 506
Genset 03 -> port 507
...
Genset 40 -> port 544
```

This is easiest for you to build and easiest for him to deploy.

Why this is better

Because you can fully develop it without touching Kubernetes.

You only need to prove:

```
Does the container start?
Do the 40 Modbus ports open?
Can a Modbus client read each simulated genset?
Can config turn gensets on/off?
Can register maps be edited from a config file?
```

Your colleague can then wrap it in Kubernetes.

What you should give him

1. Docker image

Either:

```
sim-modbus-gensets:0.1.0
```

Or a Git repo with Dockerfile.

2. Port list

```
505-544 TCP
```

3. Config file

Example:

```
YAML

gensets:
  - id: GEN01
```

```

    enabled: true
    port: 505
    unit_id: 1
    register_offset: 0

- id: GEN02
  enabled: true
  port: 506
  unit_id: 1
  register_offset: 250

```

4. Runtime variables

Example:

```

UPSTREAM_MODBUS_HOST=192.168.1.50
UPSTREAM_MODBUS_PORT=502
SIM_CONFIG=/app/config/gensets.yaml
LOG_LEVEL=INFO

```

5. Kubernetes notes

Give him a sample service like:

YAML

```

ports:
- name: gen01
  port: 505
  targetPort: 505
- name: gen02
  port: 506
  targetPort: 506

```

He can adapt it.

Development stages

Stage 1 — Static simulator

No upstream polling yet.

Each genset exposes fake values:

```

Voltage
Frequency

```

```
kw
Breaker status
Running status
Alarm bits
```

Goal: prove Modbus TCP ports work.

Stage 2 — Config-driven simulator

Values come from YAML/JSON config.

Goal: edit simulated values without changing code.

Stage 3 — Upstream mapper

Poll the existing industrial PC simulator and map values into each virtual genset.

Goal:

```
Industrial PC register block -> GEN01 port 505
Industrial PC register block -> GEN02 port 506
...
```

Stage 4 — Behavior model

Add logic:

```
If GEN01 enabled false -> close port or mark offline
If running -> frequency = 60 Hz
If breaker closed -> kW follows plant load
If trip -> breaker opens, kW drops to 0
```

What I would avoid initially

Avoid these for rev0:

```
40 Kubernetes pods
ModbusPal
ModSim GUI software
Manual simulator tools
Windows-only Modbus software
Anything that requires your colleague to reverse-engineer your setup
```

Those will create handoff friction.

Final recommendation

Build this:

```
Python + PyModbus
Dockerized
Single container
40 Modbus TCP ports
Config-driven YAML
docker-compose for your Windows testing
Kubernetes YAML example for handoff
```

That gives you a clean local development workflow and gives your colleague something deployable without needing him to understand your Python logic.



OK, I have yet another caveat that I want to take into consideration we are running inductive automations ignition HMISCADA software platform and it's already deployed and running on the same cluster and I want to use it to build a front end that would monitor and manage these python simulators what would be the best way to do that like through an API or through their SDK or something or what?

Best approach: **don't use the Ignition SDK for this yet.**

Use a **REST API on the Python simulator container**, then have Ignition call it.

Recommended architecture

```
Ignition Perspective HMI
|
| HTTP REST calls
v
Python Simulator Manager API
|
+-- Start/stop genset simulators
+-- Enable/disable GEN01-GEN40
+-- Set operating mode
```

```

    +-- Inject faults
    +-- Change load / kW / status
    +-- Return simulator health/status
    |
    v
PyModbus TCP simulator ports
505-544

```

Why REST API is the best fit

Ignition can already do HTTP calls from scripting using `system.net.httpClient()`.

So your Python app exposes endpoints like:

```

GET /api/status
GET /api/gensets
POST /api/gensets/GEN01/enable
POST /api/gensets/GEN01/disable
POST /api/gensets/GEN01/load
POST /api/gensets/GEN01/fault
POST /api/scenario/start
POST /api/scenario/stop

```

Ignition Perspective buttons call those endpoints.

What Ignition should use

Use:

```

Perspective View
  Buttons / dropdowns / toggles
    |
Gateway script / project script
    |
system.net.httpClient()
    |
Python REST API

```

Example Ignition-side concept:

Python	Run
<pre> client = system.net.httpClient() response = client.post(</pre>	

```
"http://simulator-manager:8000/api/gensets/GEN01/disable"  
)
```

What Python should expose

Use FastAPI.

Python container would run:

```
FastAPI control API on port 8000  
Modbus TCP GEN01 on port 505  
Modbus TCP GEN02 on port 506  
...  
Modbus TCP GEN40 on port 544
```

So one container exposes both:

```
8000 = simulator control API  
505-544 = Modbus TCP simulated devices
```

Ignition monitoring options

Use two paths:

1. Control/status through REST API

For simulator management:

```
online/offline  
enabled/disabled  
scenario running  
fault injected  
process model state  
simulation speed  
upstream connection status
```

2. Real Modbus reads through Ignition

For simulated device data:

```
GEN01 voltage  
GEN01 frequency  
GEN01 kW
```

```
GEN01 breaker status  
GEN01 alarm bits
```

That proves the simulator behaves like real equipment.

Avoid using SDK initially

The Ignition SDK is better for:

```
custom modules  
custom tag providers  
custom device drivers  
gateway services  
designer extensions
```

But for this use case, SDK adds friction. You don't need a custom Ignition module just to control Python simulators.

Use SDK later only if you want a polished internal product like:

```
Simulator Manager module  
custom Gateway webpage  
custom device driver  
custom Perspective components  
built-in simulator lifecycle management
```

Best rev0 design

Python simulator container:

- FastAPI control API :8000
- PyModbus genset servers :505-544
- YAML config
- JSON status endpoint
- scenario endpoints

Ignition:

- Perspective dashboard
- REST API buttons
- Modbus TCP device connections to simulator ports
- Tags bound to simulated Modbus values

Handoff to Kubernetes colleague

Tell him the container needs exposed ports:

```
8000 TCP API
505-544 TCP Modbus simulators
```

Inside Kubernetes, Ignition can call:

```
http://simulator-manager:8000/api/status
```

And connect Modbus devices to:

```
simulator-manager:505
simulator-manager:506
...
simulator-manager:544
```

Final recommendation

Build **FastAPI + PyModbus** in the same **Python** container.

Ignition should manage it over **REST API**, and validate field behavior through normal **Modbus TCP** tags.



Yes, I agree. I don't want to use the SDK if I don't have to so at a minimum what I would want with ignition is to have a real time dashboard for monitoring but also I want some control like if I want to override a simulator and stop it and close it or change parameters or if I want to launch more simulators, even if they're not needed dynamically, etc., kind of like full control to create and everything. Can we still do that without the SDK

Yes. You can do full control without the Ignition SDK.

Use this split:

```
Ignition Perspective = operator/control dashboard
Python FastAPI app = simulator control brain
```


PyModbus = simulated Modbus TCP devices
Kubernetes = runtime/deployment layer

What Ignition can control through REST API

Ignition buttons, dropdowns, tables, and forms can call API endpoints like:

```
GET /api/status
GET /api/simulators
POST /api/simulators/create
POST /api/simulators/GEN01/start
POST /api/simulators/GEN01/stop
POST /api/simulators/GEN01/delete
POST /api/simulators/GEN01/override
POST /api/simulators/GEN01/clear-override
POST /api/scenarios/start
POST /api/scenarios/stop
POST /api/scenarios/reset
```

So from Ignition you could:

```
Start GEN01
Stop GEN01
Launch GEN01-GEN15
Disable GEN07
Inject fault on GEN12
Change GEN04 load target
Force breaker closed/open
Change simulation speed
Reset all simulators
Create additional simulated device endpoint
```

Key design decision

Do **not** have Ignition directly create containers.

Have Ignition call your **Simulator Manager API**.

The API decides what happens:

```
Ignition button click
↓
POST /api/simulators/GEN05/start
↓
Python Simulator Manager
↓
```

Starts listener on port 509



GEN05 Modbus TCP server becomes available

For Kubernetes later

There are two control levels:

Level 1 — Process-level control inside one container

Best for rev0.

The Python app starts/stops individual Modbus servers internally.

One container

API :8000

GEN01 :505

GEN02 :506

GEN03 :507

Ignition can start/stop simulated devices without Kubernetes involvement.

Level 2 — Kubernetes-level control

Later, your colleague can expose API actions that call Kubernetes to scale pods.

POST /api/simulators/GEN21/create



Simulator Manager API



Kubernetes API creates pod/service



GEN21 becomes available

Ignition still only talks REST. No SDK required.

What I would build first

Python FastAPI endpoints

Minimum set:

/api/health

/api/status

/api/simulators

/api/simulators/{id}/start

```
/api/simulators/{id}/stop  
/api/simulators/{id}/override  
/api/simulators/{id}/clear-override  
/api/simulators/{id}/setpoint  
/api/scenarios/start  
/api/scenarios/stop
```

Ignition Perspective screens

Minimum dashboard:

Plant overview

- Total simulators
- Running simulators
- Stopped simulators
- Faulted simulators
- Upstream connection status

Genset table

- GEN ID
- Port
- Running/stopped
- kW
- Voltage
- Frequency
- Breaker status
- Override active
- Start/stop buttons

Control panel

- Launch range GEN01-GEN15
- Stop range
- Inject fault
- Clear fault
- Set load target
- Reset all

Monitoring path

Use both:

```
REST API = simulator management/status  
Modbus TCP tags = actual simulated device data
```

That way Ignition sees both:

Is the simulator process healthy?
Is the simulated Modbus device behaving correctly?

Final answer

Yes — full monitoring, start/stop, overrides, parameter changes, simulator creation, and scenario control can all be done without the SDK.

Use **Ignition Perspective + REST API + FastAPI + PyModbus**.

Only use the Ignition SDK later if you want to turn this into a polished packaged Ignition module.



OK now let's talk about data transfer and interfacing, etc.
so there's a single industrial server that has all of the mods
TCP day that is being simulated. How do I get that data
into each individual devices simulator so is it best to have
a two layer process where I have a layer on top of the
initial PC that grabs all of the data and then manages it to
each industrial to each device simulator or is it just an all-
in-one module that grabs the data and then and then
deploy it, etc., with each module inside of it.

Use a **two-layer design inside one application/container** for rev0.

```
Existing industrial PC Modbus simulator
    ↓
Upstream Poller / Plant Data Cache
    ↓
Device Mapper / State Model
    ↓
GEN01 Modbus server :505
GEN02 Modbus server :506
GEN03 Modbus server :507
...
GEN40 Modbus server :544
```

Best architecture

1. Upstream Poller

One process polls the existing industrial PC Modbus TCP server.

It reads the large register blocks once, then stores them in an internal cache.

```
Industrial PC :502
      ↓
Plant data cache
```

This avoids having 40 simulators all hammering the same upstream server.

2. Plant Data Cache

This is your normalized internal memory model.

Example:

```
GEN01.running
GEN01.kW
GEN01.voltage
GEN01.frequency
GEN01.breaker_closed

GEN02.running
GEN02.kW
GEN02.voltage
GEN02.frequency
GEN02.breaker_closed
```

This cache becomes the single source of truth for the simulator layer.

3. Device Mapper

This maps plant-level source registers into each virtual device's register map.

Example:

```
Upstream register 40001 → GEN01 kW
Upstream register 40002 → GEN01 voltage
Upstream register 40251 → GEN02 kW
Upstream register 40252 → GEN02 voltage
```

Or config-driven:

YAML

```

GEN01:
  port: 505
  source_offset: 0

GEN02:
  port: 506
  source_offset: 250

```

4. Device Modbus Servers

Each virtual device exposes its own Modbus TCP server.

```

GEN01 :505
GEN02 :506
GEN03 :507

```

They do **not** independently poll the industrial PC. They read from the shared cache.

Avoid this design

```

GEN01 simulator polls industrial PC
GEN02 simulator polls industrial PC
GEN03 simulator polls industrial PC
...
GEN40 simulator polls industrial PC

```

That creates:

```

duplicate traffic
timing drift
harder debugging
more failure points
inconsistent snapshots

```

Recommended rev0

One Docker container:

```

Python app
- FastAPI control API :8000
- Upstream Modbus poller
- Shared plant cache

```

- Register mapper
- PyModbus servers :505-544

This gives you the cleanest handoff.

When to split into separate services

Only split later if needed:

```
Service 1: upstream-poller
Service 2: simulator-manager-api
Service 3+: individual device simulators
```

That is better for scale, but worse for rev0 complexity.

My recommendation

Build this first:

```
All-in-one container
but internally separated into layers:
1. upstream poller
2. plant cache
3. device mapper
4. Modbus server endpoints
5. REST control API
```

So architecturally it is two-layered, but operationally it is one deployable unit.



I am currently logged in to the node in the kubernetes cluster, what command\commands should I run to help us understand what is possible on it?

Run these in order. This will tell you what you can actually do on that node and cluster.

1. Identify environment

Bash

```
whoami  
hostname  
pwd
```

2. Check Kubernetes access

Bash

```
kubectl version --client  
kubectl get nodes -o wide  
kubectl config current-context
```

If that fails:

Bash

```
sudo microk8s kubectl get nodes
```

3. Check permissions (this is critical)

Bash

```
kubectl auth can-i --list
```

Look for:

```
pods  
deployments  
services  
configmaps
```

If you don't have create/patch/delete → your colleague must deploy for you.

4. See what's already running

Bash

```
kubectl get pods -A -o wide  
kubectl get services -A  
kubectl get deployments -A
```

Look for:

- Ignition pods

- existing simulators
- namespaces in use

5. Find Ignition

Bash

```
kubectl get pods -A | grep -i ignition  
kubectl get svc -A | grep -i ignition
```

You want the service name so your simulator can talk to it (and vice versa).

6. Check networking constraints

Bash

```
kubectl get svc -A -o wide  
kubectl get endpoints -A
```

You're looking for:

- ClusterIP ranges
- how ports are exposed
- naming conventions

7. Check container runtime (important for your build)

Bash

```
docker --version
```

If not present:

Bash

```
ctr version  
crictl version
```

This tells you how images are handled.

8. Check if you can run test containers

Bash

```
kubectl run test-nginx --image=nginx --restart=Never  
kubectl get pods
```

If denied → you don't control deployments.

9. Check storage / volumes (optional but useful)

Bash

```
kubectl get pvc -A  
kubectl get storageclass
```

10. Check logs of something existing

Pick a pod:

Bash

```
kubectl logs <pod-name> -n <namespace>
```

This confirms you can inspect runtime behavior.

What you're trying to determine

After running all that, answer these:

1. Do I have permission to create pods/services?
2. What namespace should I use?
3. How are ports exposed (ClusterIP, NodePort, etc)?
4. How do services talk to each other (DNS names)?
5. Is Docker used or containerd?

What to send back

Paste results of:

Bash

```
kubectl get nodes -o wide  
kubectl auth can-i --list
```

```
kubect1 get pods -A -o wide
kubect1 get svc -A
```

Then I can tell you exactly how to structure your simulator deployment for that cluster.



```
PS C:\Users\MiguelGrillo> tsh ssh -L 9089:127.0.0.1:9088
vg@lab-node-01
vg@lab-node-01:~$ whoami
vg
vg@lab-node-01:~$ hostname
lab-node-01
vg@lab-node-01:~$ pwd
/home/vg
vg@lab-node-01:~$ kubect1 version --client
de
kubect1 config current-contextInsufficient permissions to
access MicroK8s.
You can either try again with sudo or add the user vg to
the 'microk8s' group:
```

```
sudo usermod -a -G microk8s vg
sudo chown -f -R vg ~/.kube
```

After this, reload the user groups either via a reboot or by running 'newgrp microk8s'.

```
vg@lab-node-01:~$ kubect1 get nodes -o wide
Insufficient permissions to access MicroK8s.
You can either try again with sudo or add the user vg to
the 'microk8s' group:
```

```
sudo usermod -a -G microk8s vg
sudo chown -f -R vg ~/.kube
```

After this, reload the user groups either via a reboot or by running 'newgrp microk8s'.

```
vg@lab-node-01:~$ kubect1 config current-context
```

You're on a **MicroK8s node**, but user `vg` does **not** currently have direct MicroK8s permissions.

Run this next:

Bash

```
sudo microk8s kubectl get nodes -o wide
```

Then:

Bash

```
sudo microk8s kubectl get pods -A -o wide
```

Then:

Bash

```
sudo microk8s kubectl get svc -A
```

If `sudo` works, use this alias for the session:

Bash

```
alias kubectl='sudo microk8s kubectl'
```

Then rerun:

Bash

```
kubectl get nodes -o wide  
kubectl get pods -A -o wide  
kubectl get svc -A
```

Also check runtime:

Bash

```
docker --version  
sudo microk8s ctr version
```

Don't run the `usermod` command yet unless you own/admin this node. Use `sudo microk8s kubectl` first.



i dont want to write anything just fyi, a different group manages the cluster

Then stay read-only.

Use only:

Bash

```
sudo microk8s kubectl get nodes -o wide
sudo microk8s kubectl get pods -A -o wide
sudo microk8s kubectl get svc -A
sudo microk8s kubectl get namespaces
```

Optional read-only details:

Bash

```
sudo microk8s kubectl describe pod <pod-name> -n <namespace>
sudo microk8s kubectl logs <pod-name> -n <namespace>
```

Avoid:

Bash

```
kubectl run
kubectl create
kubectl apply
kubectl delete
kubectl patch
kubectl scale
```

What we need is just discovery:

```
What namespaces exist
Where Ignition is running
```

What service name/port Ignition uses
 Whether other simulators already exist
 How services are exposed



Install the latest PowerShell for new features and improvements! <https://aka.ms/PSWindows>

Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.

```
PS C:\Users\MiguelGrillo> tsh ssh -L 9089:127.0.0.1:9088
vg@lab-node-01
```

```
vg@lab-node-01:~$ sudo microk8s kubectl get nodes -o
wide
```

```
e
```

```
sudo microk8s kubectl get svc -A
```

```
sudo microk8s kubectl get namespaces
```

```
^C
```

```
vg@lab-node-01:~$ sudo microk8s kubectl describe pod
<pod-name> -n <namespace>
```

```
-bash: syntax error near unexpected token `newline'
```

```
vg@lab-node-01:~$ sudo microk8s kubectl logs <pod-
name> -n <namespace>
```

```
-bash: syntax error near unexpected token `newline'
```

```
vg@lab-node-01:~$
```

You pasted placeholder commands literally. Don't use `<pod-name>` or `<namespace>` until we know real names.

Run just this one command first:

Bash

```
sudo microk8s kubectl get pods -A -o wide
```

Then paste the output.

Also, this line looks like it got broken:

Bash

```
sudo microk8s kubectl get nodes -o wide
e
sudo microk8s kubectl get svc -A
```

Run commands one at a time, not pasted as a block.



it just sits there:

```
vg@lab-node-01:~$ sudo microk8s kubectl get pods -A -o
wide
```

That usually means `sudo` is waiting for your password but the prompt is not showing clearly through `tsh`.

Press:

Bash

```
Ctrl+C
```

Then run:

Bash

```
sudo -v
```

If it asks for a password, enter it.

Then retry:

Bash

```
sudo microk8s kubectl get pods -A -o wide
```

If `sudo -v` also hangs, run:

Bash

```
groups
```

Then:

Bash

```
microk8s status --wait-ready
```

Do **not** wait indefinitely. If no output after ~10 seconds, `ctrl+c` and paste what happened.



```
vg@lab-node-01:~$ sudo microk8s kubectl get nodes -o  
wide
```

```
e
```

```
sudo microk8s kubectl get svc -A
```

```
sudo microk8s kubectl get namespaces
```

```
^C
```

```
vg@lab-node-01:~$ sudo microk8s kubectl describe pod  
<pod-name> -n <namespace>
```

```
-bash: syntax error near unexpected token 'newline'
```

```
vg@lab-node-01:~$ sudo microk8s kubectl logs <pod-  
name> -n <namespace>
```

```
-bash: syntax error near unexpected token 'newline'
```

```
vg@lab-node-01:~$ sudo microk8s kubectl get pods -A -o  
wide
```

```
^C
```



```
vg@lab-node-01:~$ sudo -v
vg@lab-node-01:~$ sudo microk8s kubectl get pods -A -o
wide
```

^C

```
vg@lab-node-01:~$ groups
vg adm dip plugdev users lxd wireshark
vg@lab-node-01:~$ microk8s status --wait-ready
Insufficient permissions to access MicroK8s.
You can either try again with sudo or add the user vg to
the 'microk8s' group:
```

```
sudo usermod -a -G microk8s vg
sudo chown -f -R vg ~/.kube
```

After this, reload th

vg is **not in the** microk8s **group**, and `sudo microk8s kubectl` is hanging. Since you don't want to write/change anything, don't run the `usermod` fix.

Run these read-only checks instead:

Bash

```
sudo microk8s status
```

Then:

Bash

```
sudo systemctl status snap.microk8s.daemon-kubelite --no-pager
```

Then:

Bash

```
sudo journalctl -u snap.microk8s.daemon-kubelite -n 50 --no-pager
```

Also check whether `kubectl` itself can respond with a short timeout:

Bash

```
sudo microk8s kubectl get pods -A --request-timeout=5s
```

Likely possibilities:

1. MicroK8s API server is not responding.
2. The node is not healthy.
3. sudo works, but kubectl is waiting on the Kubernetes API.
4. You only have OS access, not usable cluster access.

Most useful next command:

Bash

```
sudo microk8s status
```



If I proceed with a docker container, and I need python 3.11, do I install it within the dockert container and then 3.11 runs within the docker container on the cluser?

Yes.

You install Python 3.11 inside the Docker image, not on the cluster nodes.

How it works

```
Your laptop (Windows)
  ↓ build image
Docker image (includes Python 3.11 + your app)
  ↓ push / handoff
Kubernetes cluster
  ↓ run container
Container runs Python 3.11 internally
```

The cluster does **not** need Python installed at all.

What you do

Dockerfile (example)

dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

This line is the key:

dockerfile

```
FROM python:3.11-slim
```

That gives you a container with Python 3.11 baked in.

Verify locally (Windows)

PowerShell

```
docker build -t sim:0.1 .
docker run sim:0.1 python --version
```

You should see:

```
Python 3.11.x
```

What happens in Kubernetes

Your colleague deploys:

YAML

```
image: sim:0.1
```

Kubernetes just runs the container. Inside it:

```
Python 3.11
Your simulator code
PyModbus
FastAPI
```

Important

Do not:

```
install Python on the node
assume cluster has Python
depend on system Python
```

Everything must be inside the container.

Bottom line

```
Docker image = your runtime environment
Kubernetes = just runs it
```

So yes — Python 3.11 lives entirely inside the container.



what stack, process, etc, will allow for the least amount of friction for the kubernetes cluster admin when deploying?

Use this stack:

- Python 3.11 + FastAPI + PyModbus
- Docker container
- YAML config file
- Single container first
- One Kubernetes Deployment
- One Kubernetes Service
- One ConfigMap
- No database for rev0
- No custom Kubernetes operators
- No Ignition SDK

Least-friction handoff package

Give the cluster admin:

```
simulator/  
  Dockerfile  
  requirements.txt  
app/  
config/  
  simulator.yaml  
k8s/  
  deployment.yaml  
  service.yaml  
  configmap.yaml  
README.md
```

Deployment model

Start with:

- 1 container
- API port: 8000
- Modbus ports: 505-544

Not 40 pods yet.

That means the admin only has to deploy:

- Deployment
- Service
- ConfigMap

Best container strategy

Build and push image to whatever registry they already use:

```
registry.company.local/modbus-sim:0.1.0
```

Then Kubernetes YAML references that image.

Best config strategy

Put all simulator setup in one YAML file:

YAML

```
upstream:
  host: 192.168.x.x
  port: 502

api:
  port: 8000

gensets:
  count: 40
  base_port: 505
  enabled_default: false
```

That lets the admin change config without touching code.

Best admin experience

The admin should only need to run something like:

Bash

```
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

Or, better:

Bash

```
kubectl apply -f k8s/
```

What to avoid

Avoid for rev0:

- Helm chart
- 40 Deployments
- 40 Services
- Persistent volumes
- Databases
- Ingress
- Kubernetes API control from Python
- Ignition SDK
- Node-level installs
- Manual Python setup

My recommendation

Build it as:

- single stateless container
- config-driven
- REST-controlled
- all ports declared
- logs to stdout
- health endpoint

Required endpoints:

- GET /health
- GET /api/status
- GET /api/simulators
- POST /api/simulators/{id}/start
- POST /api/simulators/{id}/stop
- POST /api/simulators/{id}/override

That gives the Kubernetes admin the cleanest possible deployment.



so the full final version of the simulator will need to deploy and run 200+ modbus tcp server simulators each with their own unique port

would it be best to have a single application in the container that rolls everything up together? so it would be:

1) the custom application in the docker container as the modbus tcp client communicating with the master controller modbus tcp server that has the full modbus map of 200+ devices all together

2) custom application in docker then ingests all of the data from the master controller

3) custom application spawns all 200+ simulated modbus tcp server devices exposing each of their modbus ports

or have the application spawn individual instances\runtimes for each device simulator?

Thought for a couple of seconds >

For **200+ Modbus TCP server simulators**, do **not** start with 200 separate runtimes.

Best final architecture:

```
One container
One Python application
One upstream Modbus TCP client
One shared plant data cache
200+ lightweight Modbus TCP server listeners
One REST API for Ignition/control
```

Recommended model

```
Master controller Modbus TCP server
Full 200+ device map
↓
Single upstream poller/client
↓
Internal normalized cache
↓
Device mapper
↓
Port 505 Device 001 Modbus server
Port 506 Device 002 Modbus server
Port 507 Device 003 Modbus server
```


...

Port 704+ Device 200+ Modbus server

Why this is best

A Modbus TCP server listener is lightweight. You do **not** need 200 Python runtimes unless each device has heavy independent behavior.

Use one application with:

- asyncio tasks
- shared memory/cache
- per-device server objects
- per-device register blocks
- per-device state machines

Avoid this for rev0/final unless forced

- 200 containers
- 200 pods
- 200 deployments
- 200 services
- 200 Python processes

That creates unnecessary pain:

- more Kubernetes objects
- more service config
- more logs to inspect
- more health checks
- more image pulls
- more startup sequencing
- more admin friction
- harder troubleshooting

When individual instances make sense

Use individual runtimes only if you need:

- device-level crash isolation
- different simulator code per device type
- different software versions per device type
- heavy CPU simulation per device

```
independent scaling  
Kubernetes-level lifecycle per device
```

For your case, I would not do that first.

Better compromise

Use **one runtime per device class**, not one runtime per device.

Example:

```
Container 1: genset-simulators      ports 505-544  
Container 2: breaker-simulators    ports 600-680  
Container 3: relay-simulators      ports 700-760  
Container 4: syncon-simulators     ports 800-820
```

That is the best future structure if one giant container gets too busy.

My recommended progression

Rev0

```
One container  
40 gensets  
ports 505-544
```

Rev1

```
One container  
200+ devices  
ports assigned by config
```

Rev2, only if needed

```
Multiple containers split by device class  
not by individual device
```

Final answer

Build **one custom application inside one container** that:

1. Polls the master controller once
2. Ingests the full Modbus map

3. Normalizes the data into an internal cache
4. Maps data into 200+ virtual device register spaces
5. Runs 200+ Modbus TCP server listeners on unique ports
6. Exposes REST API control/status to Ignition

Do **not** spawn 200 separate runtimes unless performance testing proves you need it.



explain containerization to me, is Docker and Kubernetes two different versions of doing containers?

No. They are not two versions of the same thing.

Core idea

Containerization = packaging an application + its runtime + dependencies into a single unit that runs the same anywhere.

Roles

Docker

Builds and runs containers

- You write code
- Dockerfile defines environment (Python 3.11, libs, app)
- Docker builds an image
- You can run that image locally as a container

Example:

Bash

```
docker build -t sim:0.1 .  
docker run sim:0.1
```

Docker = **developer tool** + **container runtime**

Kubernetes

Runs and manages containers at scale

```
You give Kubernetes a container image
→ Kubernetes decides where to run it
→ Keeps it alive
→ Restarts it if it crashes
→ Exposes it on the network
→ Scales it
```

Kubernetes = **orchestrator**

Relationship

```
Docker → creates container images
Kubernetes → runs those images in a cluster
```

Kubernetes does NOT build your app.

It only runs what you give it.

Simple analogy

```
Docker = how you package and ship a machine
Kubernetes = the factory floor that runs hundreds of those machines
```

In your case

```
You (Windows):
  build Docker image with Python + simulator
```

```
Your colleague:
  deploys that image into Kubernetes
```

```
Cluster:
  runs your simulator container
```

Important clarification

Modern Kubernetes does NOT require Docker specifically.

It runs containers using runtimes like:

```
containerd
CRI-O
```

But it still runs **Docker-built images** just fine.

Bottom line

```
Docker = build + test container
Kubernetes = deploy + run container
```

They work together, not instead of each other.



so the servers that are running in the FAT that I am building this simulator for are Kubernetes clusters, so do I still need to build the application in Docker on my windows machine?

Yes.

You still build it as a Docker container on your Windows machine.

Why

Kubernetes does not run your raw Python code. It runs **container images**.

So your workflow is:

```
Your PC (Windows)
→ build container image
→ test locally
→ hand off image
```

```
Kubernetes cluster  
→ pulls image  
→ runs container
```

What happens if you don't

If you try to give them:

```
Python code only  
requirements.txt
```

Then the admin has to:

```
figure out Python version  
install dependencies  
build runtime  
debug environment issues
```

That creates friction.

What you should deliver

You give them:

```
Docker image (or Dockerfile + repo)
```

Example:

```
registry.company.local/modbus-sim:0.1.0
```

Then they deploy:

YAML

```
image: registry.company.local/modbus-sim:0.1.0
```

Done.

Your local setup

On Windows:

```
Install Docker Desktop  
Build image  
Run container locally  
Test Modbus ports + API
```

Key point

Even though Kubernetes uses containerd internally, it still runs **standard container images**, which you build with Docker.

Bottom line

```
Yes – build and test with Docker locally  
Kubernetes will run that same container image
```



is there anything I need to know about the Kubernetes install like versions etc to make sure my Docker image works with it? I may be wrong but I believe the whole idea of containerization is that after the container is built, it can run anywhere

You're mostly right, but not 100%. Containers are portable, not magic. There are a few things that can break you if you ignore them.

What you do NOT need to worry about

```
Kubernetes version (for your use case)  
Node OS version  
Whether they use Docker vs containerd  
Python installed on the cluster
```

If you build a standard image, it will run.

What you DO need to get right

1. CPU architecture

This is the biggest one.

Check what the cluster uses:

Bash

```
uname -m
```

Common:

```
x86_64 → safe (most servers)
arm64  → Apple Silicon / some edge clusters
```

If you build on Windows x86 and cluster is x86 → fine

If mismatch → your container will not run

Safe approach:

Bash

```
docker buildx build --platform linux/amd64 -t sim:0.1 .
```

2. OS base (must be Linux)

Kubernetes does NOT run Windows containers in most clusters.

Your Dockerfile must be:

dockerfile

```
FROM python:3.11-slim
```

Not:

```
windows-based images
```

3. Network ports

You are exposing 200+ ports.

Make sure:

```
Ports are declared and consistent
No conflicts
Cluster allows that many ports on a service
```

This is the one thing your cluster admin will care about.

4. Bind address

Inside container, bind to:

```
0.0.0.0
```

NOT:

```
127.0.0.1
```

If you bind to localhost, nothing outside the container can connect.

5. No local filesystem assumptions

Avoid:

```
C:\ paths
local temp files required for runtime
writing persistent state to disk
```

Containers should be stateless.

6. Logging

Write logs to:

```
stdout / stderr
```

NOT to files.

Kubernetes collects logs automatically.

7. Health endpoint

Provide:

```
GET /health
```

So they can wire readiness/liveness probes.

What might break portability

```
Wrong CPU architecture
Windows-based image
Binding to localhost only
Hardcoded IP addresses
Assuming local disk persistence
```

What you do NOT need to match

```
Kubernetes version
MicroK8s vs EKS vs AKS vs GKE
Container runtime (Docker vs containerd)
```

Bottom line

Containerization gives you:

```
Same runtime everywhere
Same dependencies everywhere
Same behavior everywhere
```

But only if you:

build for the correct architecture
use Linux base images
follow container best practices



give me the read only command that will tell us the cpu architecture

Run this:

Bash

Common outputs:

x86_64 → Intel/AMD (most servers)
aarch64 → ARM64

Optional extra detail:

Bash



yeah its x86_64

Build for:

Bash

Use this on Windows:

PowerShell

That matches x86_64 .



```
vg@lab-node-01:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
Address sizes:          39 bits physical, 48 bits virtual
CPU(s):                 8
On-line CPU(s) list:    0-7
Thread(s) per core:     1
Core(s) per socket:     8
Socket(s):              1
NUMA node(s):           1
Vendor ID:              GenuineIntel
CPU family:              6
Model:                  158
Model name:              Intel(R) Core(TM) i7-9700TE
CPU @ 1.80GHz
Stepping:               13
CPU MHz:                 800.065
CPU max MHz:             3800.0000
CPU min MHz:             800.0000
BogoMIPS:                3600.00
Virtualization:          VT-x
L1d cache:               256 KiB
L1i cache:               256 KiB
L2 cache:                2 MiB
L3 cache:                12 MiB
NUMA node0 CPU(s):       0-7
Vulnerability Gather data sampling: Mitigation; Microcode
Vulnerability Itlb multihit: KVM: Mitigation: Split huge
```

pages

Vulnerability L1tf: Not affected

Vulnerability Mds: Not affected

Vulnerability Meltdown: Not affected

Vulnerability Mmio stale data: Mitigation; Clear CPU buffers; SMT disabled

Vulnerability Retbleed: Mitigation; Enhanced IBRS

Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl and seccomp

Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization

Vulnerability Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBRSE-eIBRS SW

sequence; BHI SW loop, KVM SW

loop

Vulnerability Srbds: Mitigation; Microcode

Vulnerability Tsx async abort: Mitigation; TSX disabled

Flags: fpu vme de pse tsc msr pae mce

cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts

acpi mmx fxsr sse sse2 ss ht tm pbe

syscall nx pdpe1gb rdtscp lm constant_tsc art ar

ch_perfmon pebs bts rep_good nopl

xtopology nonstop_tsc cpuid aperfmperf pni pclmulq

dq dtes64 monitor ds_cpl vmx smx

est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 s

se4_2 x2apic movbe popcnt

tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm

3

dnowprefetch cpuid_fault epb

invpcid_single ssbd ibrs ibpb stibp ibrs_enhanced tpr_s

hadow vnmi flexpriority ept vpid

ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms

invpcid mpx rdseed adx smap

clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dthe

rm ida arat pln pts hwp hwp_notify

hwp_act_window hwp_epp md_clear flush_l1d arch_ca

pabilities

vg@lab-node-01:~\$

You're good. Nothing in that output blocks your plan.

What matters

```
Architecture: x86_64
```

That means:

```
Build: linux/amd64
```

```
Use: python:3.11-slim (or similar)
```

No cross-platform issues.

What this also tells you

```
8 CPUs
```

```
No SMT (1 thread per core)
```

So your simulator capacity is fine.

Running:

```
200+ Modbus TCP listeners
```

```
1 poller
```

```
1 API server
```

will not be a problem.

What to do next (concrete)

Build like this:

PowerShell

```
docker buildx build --platform linux/amd64 -t modbus-sim:0.1.0 .
```

Inside your app:

```
Bind all servers to 0.0.0.0
```

```
Use asyncio (not threads)
```

Single upstream poller

Shared cache

Optional optimization (not required now)

Later you can:

Pin poller loop to ~100-500 ms cycle

Throttle update frequency

Bottom line

No special Kubernetes tuning needed

No architecture issues

Proceed with standard Docker build



Sources